Allocate, Load, Install, Launch, and Analyze

High Performance Computing Hands-On Tutorial

Ali Lotfi
Department of Plant Sciences, University of Saskatchewan

April 2, 2026

Contents

1 What Is an HPC Cluster? 6
1.1 The anatomy of a cluster . . . 6

2 Logging In via SSH 7
2.1 Step-by-step login . . . 7
2.2 Reading the prompt . . . 8

3 Finding Your Way Around 8
3.1 Where am I? — `pwd` . . . 9
3.2 Who am I? — `whoami` . . . 9
3.3 What is here? — `ls` . . . 9
3.4 Useful flags for ls . . . 10
3.5 Moving between folders — `cd` . . . 11
3.6 Getting help — `man` and `--help` . . . 12
3.7 Which node are you on? — `hostname` . . . 13

4 Creating, Moving, and Deleting Files 13
4.1 Creating a folder — `mkdir` . . . 13
4.2 Editing files — `nano` . . . 13
4.3 Reading files . . . 14
4.4 Moving and renaming — `mv` . . . 15
4.5 Downloading files from the internet . . . 15
4.6 Copying — `cp` . . . 16
4.7 Deleting — `rm` . . . 16
4.8 Chaining commands . . . 17

5 Wildcards, Redirection, and Pipes 17
5.1 Wildcards — `*` . . . 17
5.2 echo . . . 19
5.3 Output redirection . . . 19
5.4 stdout, stderr, and stdin . . . 19
5.5 Searching inside files — `grep` . . . 20
5.6 Pipes — `|` . . . 20
5.7 Compressing and archiving . . . 20

6 Shell Scripts, Variables, and Loops 21
6.1 Writing your first script . . . 22
6.2 Running a script . . . 22
6.3 File permissions . . . 23
6.4 Variables . . . 23
6.5 For loops . . . 24

7 Submitting Jobs with SLURM **25**
7.1 The difference between running and submitting . . . 26
7.2 Writing a SLURM job script . . . 27
7.3 Monitoring and cancelling jobs . . . 28
7.4 What happens when the time limit is exceeded . . . 29
7.5 Running a single command on a compute node — `srun` . . . 30
7.6 Checking job efficiency with seff and sacct . . . 30

8 Loading Software with Modules **31**
8.1 Exploring available software . . . 31
8.2 Loading and unloading . . . 31

9 Running Python on the Cluster **32**
9.1 Step 1 — Create the environment . . . 33
9.2 Step 2 — Activate and install packages . . . 33
9.3 Step 3 — Write a Python script . . . 34
9.4 Step 4 — Write the job script . . . 34
9.5 Step 5 — Submit and check output . . . 35

10 Transferring Files with scp **35**
10.1 Uploading to the cluster . . . 36
10.2 Downloading from the cluster . . . 36
10.3 Windows line endings — dos2unix . . . 36
10.4 Large transfers — bundle first with tar . . . 37

11 Running GPU Jobs **38**
11.1 Step 1 — Find available GPUs . . . 38
11.2 Step 2 — CPU vs GPU accounts . . . 40
11.3 Step 3 — Install PyTorch and write the GPU job script . . . 40
11.4 Step 4 — Confirm the GPU is being used . . . 43
11.5 Real result — CIFAR-10 on Fir . . . 44

How to Use These Notes

These notes are a **tutorial companion**, not a reference manual. Each section follows this structure:

1. **Goal** — what problem we are solving and why
2. **Intuition** — the mental model you need before touching the keyboard
3. **Commands** — what to type, with annotated explanations
4. **Real output** — exactly what you should see on Fir
5. **Watch out for** — common mistakes from real sessions

Work through the sections in order. Every command was tested live on the Fir cluster.

Before You Begin — Substitute Your Own Username

Throughout these notes, example paths and usernames are shown that you must substitute with your own. **Wherever you see the placeholders below, replace them before typing the command.**

Substitute these everywhere before typing

```
yourUsername           -> your Alliance username (find with:
   whoami)
/scratch/yourUsername -> /scratch/YOUR_ACTUAL_USERNAME
def-kanj241_cpu        -> def-yourPI_cpu
def-kanj241_gpu        -> def-yourPI_gpu
```

Example: if your Alliance username is `jsmith`, then `/scratch/yourUsername/` becomes `/scratch/jsmith/`.

Set up your working directory

Before starting the exercises, create a working directory on scratch. **If you haven't logged in yet, go to Section 2 first, then come back here.**

Try it yourself

```
$ ssh yourUsername@fir.alliancecan.ca
$ cd ~/scratch
$ mkdir hpc-tutorial
$ cd hpc-tutorial
```

```
$ pwd
/scratch/yourUsername/hpc-tutorial
```

Use this directory for all the exercises in these notes. Scratch is fast storage, ideal for active work — just remember it is purged after 60 days, so move important results to `projects/` when you are done.

1 What Is an HPC Cluster?

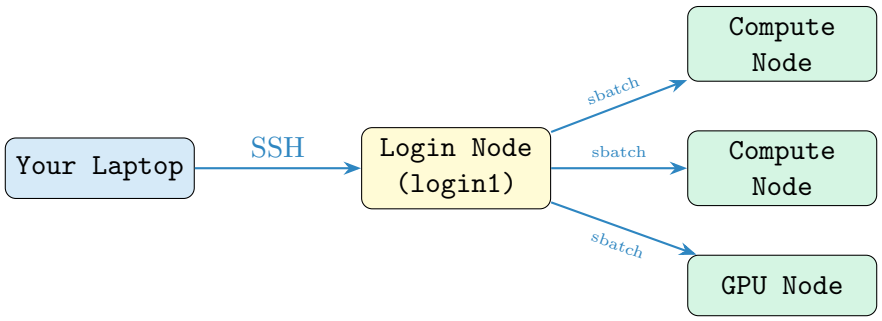
Goal of this Section

Understand *what* Fir is before we log in, so nothing surprises you.

Intuition

Your laptop has one CPU, maybe 16 GB of RAM, and one GPU. That is fine for small experiments. But what if your Python script needs to train on 100,000 images and would take 3 weeks on your laptop?
An HPC cluster is a large collection of computers — called **nodes** — connected by a fast network and managed as one system. Instead of waiting 3 weeks, you submit your job to the cluster and it runs on a powerful machine overnight.

1.1 The anatomy of a cluster



There are two types of nodes you need to know:

- **Login node** (`login1` or `login3`) — where you land when you SSH in. This is a shared gateway. You can edit files, transfer data, and submit jobs here — but *never run heavy computation* here.
- **Compute nodes** (e.g. `fc30519`) — where the real work happens. You never SSH into these directly; you reach them by submitting a SLURM job.

Rule #1

Never run Python scripts, training loops, or data processing directly on the login node. Always use `sbatch`.

2 Logging In via SSH

Goal of this Section

Connect from your Windows laptop to the Fir cluster for the first time.

Intuition

SSH (Secure SHell) creates an encrypted tunnel between your laptop and Fir over the internet. Think of it as a secure phone call: once connected, everything you type is sent to Fir and the response is sent back to your screen.

What does `ssh` stand for?

Secure **Sh**ell — "shell" refers to the command-line interface, and "secure" means the connection is encrypted.

2.1 Step-by-step login

Open **Command Prompt** on Windows. SSH is built into Windows 10/11 — no extra software needed.

Try it yourself

Type the following command (replace `yourUsername` with your Alliance username — *not* your university ID):

```
ssh yourUsername@fir.alliancecan.ca
```

You will see this sequence:

```
(yourUsername@fir.alliancecan.ca) Password:
(yourUsername@fir.alliancecan.ca) Duo two-factor login

Enter a passcode or select one of the following options:
1. Duo Push to iOS

Passcode or option (1-1): 1
Success. Logging you in...
```

```
[yourUsername@login1 ~]$
```

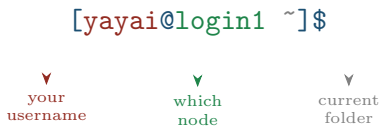
Authentication steps:

1. Type your Alliance password (characters are invisible — this is normal)
2. Type `1` and press Enter to request a Duo Push
3. Approve on your phone

You are logged in when you see the **prompt**: `[username@login1 ~]$`

2.2 Reading the prompt

The prompt is a status bar that tells you three things at a glance:



Common mistake

Using your university ID (e.g. `all054`) instead of your Alliance username gives a silent "Connection closed" with no error message. Always use your Alliance username.

Two terminals

Keep two Command Prompt windows open at all times: one SSH session on the cluster, one local window for `scp` file transfers. This workflow will save you a lot of reconnecting.

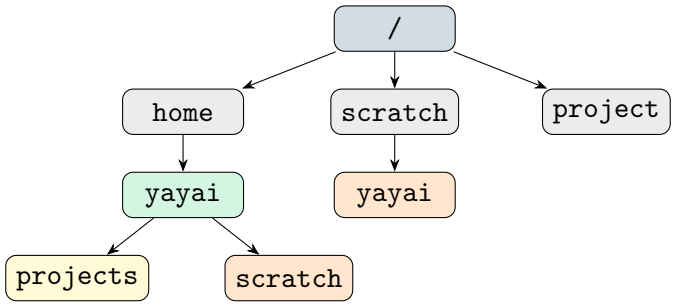
3 Finding Your Way Around

Goal of this Section

Learn to navigate the cluster filesystem confidently — know where you are, what is around you, and how to move between locations.

Intuition

The Linux filesystem is a single giant tree. Everything — files, folders, devices — hangs from one root called `/`. Your personal space is a branch of that tree at `/home/yourUsername`. Think of it like a filing cabinet: the cabinet is `/`, your drawer is `/home/yourUsername`, and your folders are inside that drawer.



3.1 Where am I? — pwd

What does pwd stand for?

Print **W**orking **D**irectory — "working directory" means the folder you are currently inside.

Try it yourself

```
$ pwd
/home/yourUsername
```

When you first log in, you always land in your **home directory**.

3.2 Who am I? — whoami

Try it yourself

```
$ whoami
yourUsername
```

Confirms which user you are logged in as. Useful when something permissions-related goes wrong.

3.3 What is here? — ls

What does ls stand for?

Li**S**t — lists the contents of a directory.

Try it yourself

```
$ ls
nearline  projects  scratch
```

Your home directory on Fir has three important locations:

Folder	Purpose	Backed up?
`projects/`	Main working storage. Put your code and results here.	Yes
`scratch/`	Fast storage for active jobs. Temporary.	No (purged after 60 days)
`nearline/`	Long-term archival tape storage.	Yes

scratch is not a real folder

When you run `ls -l`, you will see:

```
lrwxrwxrwx 1 root root 14 Feb 10 scratch ->
   /scratch/yourUsername
```

The `l` at the start and the `->` arrow mean `scratch` is a **symbolic link** (a shortcut) pointing to `/scratch/yourUsername`. Typing `cd scratch` and `cd /scratch/yourUsername` take you to the exact same place.

3.4 Useful flags for ls

Flags modify a command's behaviour. They follow the command after a dash:

```
$ ls -l       # long listing: permissions, owner, size, date
$ ls -lh      # same, but file sizes are human-readable (4.0K vs
   4096)
$ ls -a       # show ALL files, including hidden ones (starting
   with .)
$ ls -la      # combine -l and -a
$ ls -r       # list in reverse order
$ ls -t       # list sorted by last modified time (newest first)
$ ls -R       # list recursively (all subdirectories)
```

Hidden files

Any file whose name starts with a dot (`.`) is hidden from plain `ls`. These are usually configuration files like `.bashrc`. They are not hidden for security — just to keep the directory tidy.

/scratch is world-readable

Running `ls /scratch` shows folder names for **every user** on the cluster. You cannot read other users' files, but you can see their usernames. This is normal HPC behaviour — all clusters work this way.

3.5 Moving between folders — `cd`

What does `cd` stand for?

Change **D**irectory — moves you into a different folder.

```
$ cd projects/        # enter the projects folder
$ cd ..               # go up one level (parent directory)
$ cd ~                # go back to your home directory
$ cd                  # also goes home (same as cd ~)
$ cd /                # go to the root of the entire filesystem
$ cd /scratch/yourUsername  # go anywhere using a full absolute
   path
```

Intuition

Think of `..` as the "up" button in a file browser. `.` refers to the folder you are currently in. is a shortcut for your home directory `/home/yourUsername`.

Absolute vs relative paths:

Absolute path	Relative path
Starts with `/`	No leading `/`
Works from anywhere	Relative to where you are
`/home/yourUsername/projects/`	`projects/scripts/`

cd.. is not the same as cd ..

You must put a space between `cd` and `...` Without the space, the shell tries to find a command called `cd..` — which does not exist.

```
$ cd..          # ERROR: cd..: command not found
$ cd ..         # CORRECT
```

Tab autocomplete

Press `Tab` while typing any path or command to let the shell finish it for you. If there are multiple matches, press `Tab` twice to see them all. This is one of the most useful habits you can develop.

3.6 Getting help — `man` and `--help`

Every command has a manual:

```
$ man ls            # opens the full manual (press q to quit)
$ ls --help         # prints a quick summary
```

Angle brackets in documentation

When a tutorial shows `man <command>`, the angle brackets are **placeholders**. Do not type them. Replace the whole `<command>` with the actual command name, e.g. `man ls`.

Typos in paths give clear errors

```
$ cd ,,
-bash: cd: ,,: No such file or directory
```

The shell will never silently do the wrong thing if a path doesn't exist — it will tell you. Always check your prompt after `cd` to confirm where you ended up.

Exploring the root filesystem

```
$ cd /
$ ls
afs  boot  cvmfs  home  project  scratch  nearline  ...
```

The root `/` contains the entire filesystem. On Fir you will see HPC-specific folders not found on a regular computer: `/project`, `/scratch`, `/nearline`, and `/cvmfs` (where all the software modules live).

3.7 Which node are you on? — `hostname`

Try it yourself

```
$ hostname
login1
```

You may sometimes see `login3` instead of `login1`. Fir has multiple login nodes and assigns you one automatically. Your files are always the same — the filesystem is shared.

4 Creating, Moving, and Deleting Files

Goal of this Section

Learn to create folders, create and edit text files, move and copy things around, and delete what you no longer need.

Intuition

On your laptop you use a file browser (clicking and dragging). On the cluster, you do the same things but with typed commands. Once you know the four core commands — `mkdir`, `nano`, `mv`, `cp` — you can do everything a file browser can do, but faster.

4.1 Creating a folder — `mkdir`

What does `mkdir` stand for?

Make **Dir**ectory

Try it yourself

```
$ mkdir hpc-test
$ ls
hpc-test  nearline  projects  scratch
```

4.2 Editing files — `nano`

`nano` is the simplest terminal text editor. It opens a file for editing (and creates it if it does not exist yet):

Try it yourself

```
$ nano draft.txt
```

Type some text. Then use these keyboard shortcuts:

Shortcut	What it does
`Ctrl+O`	Save the file (press Enter to confirm filename)
`Ctrl+X`	Exit nano
`Ctrl+K`	Cut the entire current line
`Ctrl+U`	Paste the cut line

Mouse does not work in nano

Clicking does not move your cursor. Use the **arrow keys** to navigate. There is no click-and-drag selection.

When to use nano

Use nano only for quick edits directly on the cluster — for example, fixing a path in a job script. Write all your Python code locally in PyCharm and transfer it with `scp`.

4.3 Reading files

```
$ cat draft.txt          # print the whole file to screen
$ cat file1 file2        # print multiple files in sequence
   (concatenate)
$ head draft.txt         # print the first 10 lines
$ head -n 20 draft.txt   # print the first 20 lines
$ tail draft.txt         # print the last 10 lines
$ less draft.txt         # scroll through the file (q to quit)
```

What does `cat` stand for?

Catenate — the original purpose was to join multiple files together. Today it is also used simply to print a file.

Never cat a large file

Running `cat` on a large file floods your terminal with thousands of lines. Use `head`, `tail`, or `less` for large files. If you accidentally run `cat` on a huge file, press `Ctrl+C` to stop it immediately.

Common cat mistake

```
$ cat file1.txt cat file2.txt   # WRONG: tries to open a file
   called "cat"
$ cat file1.txt file2.txt       # CORRECT: prints both files
   in sequence
```

4.4 Moving and renaming — mv

What does mv stand for?

Move

mv is used for both moving files *and* renaming them — it is the same command:

```
$ mv draft.txt paper/       # move INTO the paper folder
$ mv draft.txt final.txt    # rename draft.txt to final.txt
```

Always use a trailing slash when moving into a folder

```
$ mv draft.txt paper    # WRONG if paper folder doesn't exist
   yet:
                        # renames draft.txt to a file called
                           "paper"!
$ mv draft.txt paper/   # CORRECT: moves INTO the folder
```

The safest pattern is to create the folder first, then move:

```
$ mkdir paper && mv draft.txt paper/
```

mv gives **no output** on success. Silence means it worked.

4.5 Downloading files from the internet

```
$ wget https://example.com/file.tar.gz    # download a single
   file
$ curl -O https://example.com/file.tar.gz # alternative
   downloader
$ git clone https://github.com/user/repo  # clone a Git
   repository
```

wget vs git clone — an important distinction

```
# WRONG - downloads the GitHub webpage as an HTML file
$ wget https://github.com/username/my-repo.git

# CORRECT - clones the actual repository
$ git clone https://github.com/username/my-repo.git
```

`wget` downloads a single file from a direct URL. Pointing it at a GitHub repo page downloads the HTML webpage, not your code. Always use `git clone` for Git repositories.

4.6 Copying — `cp`

```
$ cp final.txt backup.txt      # copy to same folder with new
   name
$ cp final.txt paper/          # copy into another folder
$ cp -r myfolder/ destination/ # copy an entire folder (-r =
   recursive)
```

4.7 Deleting — `rm`

What does `rm` stand for?

Remove

```
$ rm filename          # delete a file
$ rm -i filename       # delete with a confirmation prompt
$ rm -r foldername     # delete a folder and everything inside it
$ rm -rf foldername    # force delete (needed for Git repos)
$ rm -f filename       # force delete single file, no
   confirmation
$ rmdir foldername     # delete an empty folder only
```

No Recycle Bin on Linux

When you delete a file with `rm`, it is gone **permanently and immediately**. There is no undo, no Recycle Bin, no recovery. Always run `ls` first to confirm you are in the right folder before deleting anything.
Use `rm -i` when unsure — it asks before each deletion.

Why rm -rf for Git repos?

Git repositories contain write-protected files in their `.git` folder. Running `rm -r` on a repo asks for confirmation on each write-protected file. The `-f` (force) flag skips these prompts. `rm -rf` is the standard way to delete a cloned repository.

4.8 Chaining commands

You can run two commands on one line:

```
$ cmd1 && cmd2     # run cmd2 ONLY if cmd1 succeeded
$ cmd1 ; cmd2      # run cmd2 regardless of whether cmd1 worked
```

Intuition

Think of `&&` as "and then, but only if that worked." Use it whenever the second command depends on the first. For example:

```
$ mkdir paper && mv draft.txt paper/   # safe
$ cd paper && rm final.txt             # rm only runs if cd
   worked
```

5 Wildcards, Redirection, and Pipes

Goal of this Section

Learn to act on many files at once, save command output to files, and chain commands together into powerful pipelines.

Intuition

So far every command has acted on one file at a time. But what if you have 16 files and want to process all of them? Typing each filename would be tedious and error-prone. Wildcards, redirection, and pipes are the three tools that make the shell genuinely powerful.

5.1 Wildcards — *

The * character is a wildcard that matches any number of characters in a filename.

First, download and extract the demo files we will use:

Try it yourself

```
$ cd ~/scratch/hpc-tutorial
$ wget
   https://cbe453.github.io/arc-plato-intro/files/bash-lesson.tar.gz
$ tar -xzvf bash-lesson.tar.gz
$ ls *.fastq | head -5
SRR307023_1.fastq
SRR307023_2.fastq
SRR307024_1.fastq
SRR307024_2.fastq
SRR307025_1.fastq
```

You now have 16 `.fastq` files to practise with.

Now try wildcards:

```
$ ls *.txt              # matches all .txt files
$ wc -l *.fastq         # count lines in all .fastq files at once
$ mkdir fastq && mv *.fastq fastq/   # create folder, then move
   all .fastq files
$ wc -l *               # match every file in the current directory
```

Intuition

The shell *expands* the wildcard before running the command. So `wc -l *.fastq` becomes `wc -l SRR307023_1.fastq SRR307023_2.fastq ...` automatically.

What does `wc` stand for?

Word **C**ount — despite the name, it also counts lines (`-l`), characters (`-c`), and words (`-w`).

No matches = literal string

If a wildcard finds no matching files, the shell passes the pattern as a literal string:

```
$ ls *.fq
*.fq    # ERROR: bash treats it as a filename, not a pattern
```

If you see this, check your file extension. Use `ls` first to confirm your wildcard matches real files before putting it in a script.

5.2 echo

What does `echo` stand for?

Echo — prints text back to the screen, like an echo.

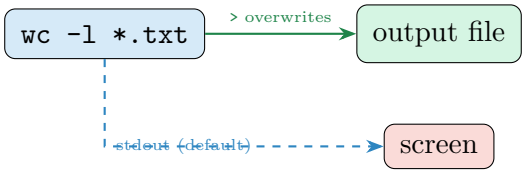
```
$ echo "Hello Fir!"              # print text to screen
$ echo -e "line1\nline2"         # \n = newline (needs -e flag)
```

Note

Without the `-e` flag, `\n` is printed literally as the two characters \and n, not as a newline.

5.3 Output redirection

By default, commands print to your screen. You can redirect this output to a file:



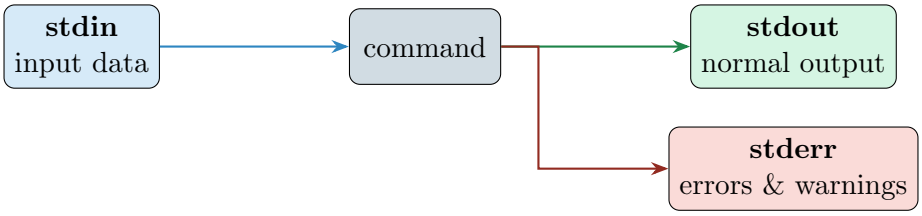

```
$ wc -l *.txt > counts.txt       # write output to file
   (OVERWRITES!)
$ wc -l *.txt >> counts.txt      # append to file (preserves old
   content)
$ wc -l * &> counts.txt          # redirect both output AND errors
$ wc -l * 2> errors.txt          # redirect only errors
```

> overwrites silently

Running > on an existing file **destroys its contents without asking**. Use >> when you want to add to a file rather than replace it.

5.4 stdout, stderr, and stdin

Every Linux command has three data streams:



- > captures only **stdout** — errors still appear on screen

- `&>` captures both **stdout and stderr**

SLURM output files on DRAC

By default on DRAC/Fir, SLURM writes **both stdout and stderr** into a single file:

```
slurm-JOBID.out     # contains print statements AND error
   messages
```

When a job fails, look for error messages near the end of `slurm-JOBID.out`. If you want to separate them, add `#SBATCH --error=errors.txt` to your job script.

5.5 Searching inside files — grep

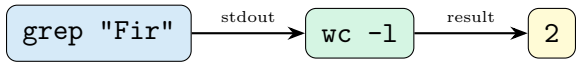
What does grep stand for?

Global **R**egular **E**xpression **P**rint — searches for a pattern and prints matching lines.

```
$ grep "Fir" text1.txt          # find all lines containing "Fir"
$ grep "Fir" *.txt              # search across all .txt files
$ grep "Act5C" genome.gtf | wc -l   # count matching lines
```

5.6 Pipes — |

The pipe operator `|` takes the output of the left command and feeds it as input to the right command:




```
$ grep "Fir" text1.txt | wc -l      # count lines matching "Fir"
$ cat output.txt | head -n 5        # show first 5 lines of
   output
$ wc -l *.fastq | sort              # count lines, then sort
```

5.7 Compressing and archiving

```
$ gzip file.txt             # compress a file -> file.txt.gz
                            # WARNING: original file is deleted!
$ gunzip file.txt.gz        # decompress back to file.txt
$ zcat file.txt.gz          # read compressed file without
   extracting
$ unzip file.zip            # extract a .zip archive
$ tar -czvf archive.tar.gz folder/   # compress a whole folder
$ tar -xzvf archive.tar.gz           # extract
```

gzip deletes the original

Unlike `cp` or `mv`, `gzip` **replaces** the original file with the compressed version. If you run `gzip mydata.txt`, the original `mydata.txt` disappears and only `mydata.txt.gz` remains. Use `gunzip` to get it back.

zcat works on files only

`zcat` reads compressed files without extracting them. It works on `.gz` files only — not on folders or other archive types.
Never run `wc -l` on a compressed file directly:

```
$ wc -l file.txt.gz      # WRONG: counts binary bytes,
   meaningless
$ zcat file.txt.gz | wc -l  # CORRECT: decompress then count
```

tar flag meanings:

Flag	Meaning
`c`	**C**reate a new archive
`x`	E**x**tract an archive
`z`	Use g**z**ip compression
`v`	**V**erbose — show files as they are processed
`f`	The next argument is the **f**ilename

Note

The `.tar.gz` extension is **not** added automatically — you must type it. Always include it so the filename makes clear what the file is.

6 Shell Scripts, Variables, and Loops

Goal of this Section

Write reusable shell scripts, use variables to store information, and loop over files automatically. This section is the direct foundation for writing SLURM job scripts.

Intuition

Instead of typing the same sequence of commands every day, you can save them in a text file (a **shell script**) and run them all at once. A SLURM job script is exactly this: a shell script with some extra lines at the top that tell SLURM how many resources to

use.

6.1 Writing your first script

Create a file called `demo.sh`:

Try it yourself

```
$ nano demo.sh
```

Type this inside:

```
#!/bin/bash

# This is a comment - it won't run
echo "Our script worked!"
```

The first line `#!/bin/bash` is required. It is called the **shebang** and tells the system to run this script using the Bash shell. Without it, the script may not behave correctly.

6.2 Running a script

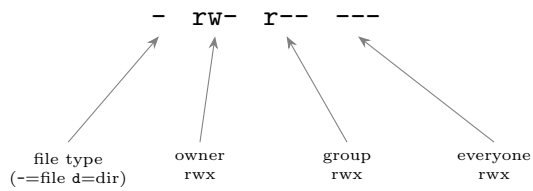
Try it yourself

```
$ sh demo.sh          # works - passes script to sh directly
Our script worked!

$ bash demo.sh        # same, using bash explicitly
Our script worked!

$ ./demo.sh           # fails without permission
-bash: ./demo.sh: Permission denied

$ chmod +x demo.sh    # add execute permission
$ ./demo.sh           # now works
Our script worked!
```

Method	Needs chmod +x?	Use when
`sh script.sh`	No	Quick test
`bash script.sh`	No	Quick test
`./script.sh`	Yes	Running your own scripts
`sbatch script.sh`	No	Submitting to SLURM

6.3 File permissions

Running `ls -l` shows a permissions string:



- **r** (read = 4) — view the file contents
- **w** (write = 2) — modify the file
- **x** (execute = 1) — run the file as a program

These permissions are **independent**. You can edit a file in nano (uses `w`) without being able to run it (needs `x`). `chmod +x` adds execute permission while leaving read and write unchanged.

```
$ chmod +x script.sh     # add execute for all users
$ chmod 600 file         # only owner can read/write; no one else
$ chmod 777 file         # everyone can read/write/execute
$ touch newfile.txt      # create an empty file
```

6.4 Variables

```
VAR="Hello from Fir"     # set a variable (no spaces around =)
echo $VAR                # use a variable (always with $)
FILES=$(ls)              # store command output in a variable
echo $FILES
```

Warning

```
wc -l FILE      # WRONG: treats FILE as a literal filename
wc -l $FILE     # CORRECT: uses the value stored in FILE
```

Always use `$` when you want the *value* of a variable. Without it, bash sees the word literally.

Wrong way to capture command output

```
TEST = ls -l     # WRONG: bash tries to run "ls" as a command
                 # -bash: -l: command not found
```

```
FILES=$(ls)     # CORRECT: wrap command in $()
```

There must be **no spaces** around the = sign when setting a variable, and command output must be wrapped in $().

Script arguments let you pass information when calling the script:

```
#!/bin/bash
wc -l $1    # $1 = first argument passed to the script
```

```
$ ./demo.sh gene_association.fb    # $1 = gene_association.fb
106290 gene_association.fb
```

Variable	Meaning
$1, $2	First, second argument
$@	All arguments at once
$0	The script name itself

Variable concatenation lets you build filenames automatically:

```
FILE=gene_association.fb
echo ${FILE}.counts           # preview:
   gene_association.fb.counts
wc -l $FILE > ${FILE}.counts  # actually create it
```

Use curly braces ${FILE} when adding text directly after the variable name so bash knows where the variable name ends.

6.5 For loops

```
#!/bin/bash

for FILE in fastq/*.fastq
do
    echo "Processing $FILE"
    wc -l $FILE > ${FILE}.counts
done
```

Intuition

The loop runs once for each item in the list after `in`. Each time through, `$FILE` is set to the next item. This creates a `.counts` file for every `.fastq` file automatically.

Common loop mistakes

- Missing `#!/bin/bash` — script may not run correctly
- `wc -l FILE` instead of `wc -l $FILE` — treats FILE as a literal name
- Wrong extension (`*.fq` instead of `*.fastq`) — no error, just prints the pattern literally
- `for`, `do`, and `done` must each be on their own line

Loop over all arguments passed to the script:

Open `demo.sh` again with `nano demo.sh` and replace its contents with:

```
#!/bin/bash
for FILE in $@
do
    wc -l $FILE
done
```

Now run it on all the `.fastq` files at once:

```
$ ./demo.sh fastq/*.fastq    # runs on all matching files at once
```

Connection to SLURM

A SLURM job script is simply a shell script with `#SBATCH` directives added at the top. Everything covered here — `#!/bin/bash`, variables, loops, comments — applies directly to writing job scripts.

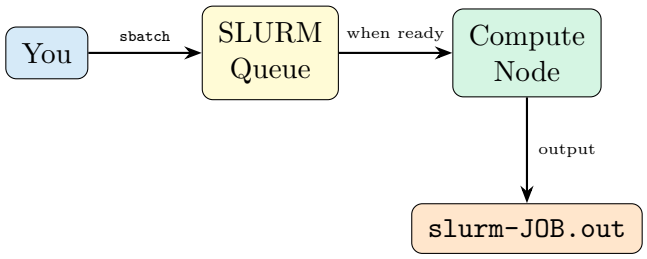
7 Submitting Jobs with SLURM

Goal of this Section

Submit your first job to a compute node, monitor it, and understand what happens when resources run out.

Intuition

The cluster is shared by thousands of users. Without a scheduler, everyone would fight over the same machines. SLURM is the "restaurant manager" — you tell it what you need (CPUs, memory, time), it puts you in a queue, and assigns you a table (compute node) when one becomes available.



7.1 The difference between running and submitting

First, make sure you are in your working directory, then create a simple job script to test with:

```
$ cd /scratch/yourUsername/hpc-tutorial
$ nano example-job.sh
```

```
#!/bin/bash
#SBATCH -J my_test_job
#SBATCH --time=00:05:00
#SBATCH --mem=256M
#SBATCH --ntasks=1
#SBATCH --account=def-kanj241_cpu

echo -n "This script is running on "
hostname
echo "Job done!"
```

Save with `Ctrl+O`, press Enter, then exit with `Ctrl+X`.

Try it yourself

Now run it *directly* and note where it runs:

```
$ sh example-job.sh
This script is running on login3    # <-- WRONG: login node!
```

Now submit it properly through SLURM:

```
$ sbatch example-job.sh
Submitted batch job 30498427
```

Check the output after it finishes:

```
$ cat slurm-30498427.out
This script is running on fc30519   # <-- CORRECT: compute
   node!
Job done!
```

7.2 Writing a SLURM job script

A SLURM job script is a shell script with `#SBATCH` directives at the top:

```
#!/bin/bash
#SBATCH -J my_test_job
#SBATCH --time=00:05:00
#SBATCH --mem=1G
#SBATCH --ntasks=1
#SBATCH --account=def-kanj241_cpu

echo -n "This script is running on "
hostname
echo "Submitted from: $SLURM_SUBMIT_DIR"
echo "Job done!"
```

$SLURM_SUBMIT_DIR

This is an environment variable automatically set by SLURM to the directory you ran `sbatch` from. It is useful for confirming that your job is finding files in the right place.

Key #SBATCH directives:

Directive	Meaning
`-J name`	Job name shown in the queue
`--time=HH:MM:SS`	Maximum walltime. Job is killed if exceeded.
`--mem=XG`	Memory requested (e.g. `1G`, `8G`)
`--ntasks=1`	Number of parallel tasks
`--account=def-kanj241_cpu`	Your PI's CPU allocation account
`--cpus-per-task=N`	CPU cores per task

$SBATCH vs #SBATCH

```
$SBATCH --time=00:05:00     # WRONG: treated as empty
   variable, ignored!
#SBATCH --time=00:05:00     # CORRECT: read by SLURM as a
   directive
```

Using `$` instead of `#` is a silent mistake — SLURM ignores the line and uses defaults.

No blank lines between #SBATCH directives

SLURM stops reading directives as soon as it hits a blank line. Any `#SBATCH` line after a blank line is silently ignored:

```
#SBATCH --time=00:05:00
                            <- blank line: SLURM stops reading
                               here!
#SBATCH --mem=1G            <- silently ignored
#SBATCH --account=def-kanj241_cpu  <- silently ignored
```

This is a common problem when editing scripts in Windows Notepad, which adds blank lines on paste. Keep all `#SBATCH` directives together at the top with no blank lines between them.

7.3 Monitoring and cancelling jobs

```
$ sbatch example-job.sh         # submit
Submitted batch job 30498427

$ squeue -u yourUsername                # check your jobs
JOBID      NAME         ST  TIME_LEFT  NODELIST
30498427   my_test_job  PD  5:00       (Priority)

$ scancel 30498427              # cancel a specific job
$ scancel -u yourUsername               # cancel ALL your jobs
```

Job states:

State	Code	Meaning
Pending	`PD`	Waiting in queue for a node
Running	`R`	Executing on a compute node
Completed	—	Finished; disappears from queue
Timeout	—	Killed by SLURM at time limit
Cancelled	—	Manually cancelled with `scancel`

Queue reasons

- `(Priority)` — other jobs have higher priority; you are waiting in line
- `(Resources)` — not enough free nodes right now
- `(None)` — just entered the queue

No output file for cancelled pending jobs

If you cancel a job while it is still `PD`, no output file is created — the job never ran on a compute node, so there is nothing to write.

7.4 What happens when the time limit is exceeded

Try it yourself

Create a job script that requests 1 minute but sleeps for 2 minutes:

```
$ nano timeout-test.sh
```

```
#!/bin/bash
#SBATCH -J timeout_test
#SBATCH --time=00:01:00
#SBATCH --mem=256M
#SBATCH --ntasks=1
#SBATCH --account=def-kanj241_cpu

echo -n "This script is running on "
hostname
sleep 120
echo "This script finished successfully."
```

```
$ sbatch timeout-test.sh
```

After 1 minute, check the output:

```
$ cat slurm-30503950.out
This script is running on fc30613
slurmstepd: error: *** JOB 30503950 ON fc30613
CANCELLED AT 2026-03-29T12:48:16 DUE TO TIME LIMIT ***
```

Notice that **"This script finished successfully." never printed** — SLURM killed the job before it got there. This is why the last `echo` in every job script acts as a **success indicator**: if you see it, the job completed.

Time limit strategy

Estimate how long your job takes, then add a 50% buffer. If unsure, request more time — unused time costs you nothing. For long jobs, save checkpoints periodically in your Python script so progress is not lost if the job times out.

```
#SBATCH --time=00:30:00    # 30 minutes
#SBATCH --time=02:00:00    # 2 hours
#SBATCH --time=1-00:00:00  # 1 day
```

7.5 Running a single command on a compute node — `srun`

For a quick one-off command that needs a compute node but doesn't need a full job script:

```
$ srun hostname           # run hostname on a compute node
fc30519

$ srun -n 2 echo "hello"  # run on 2 CPUs
hello
hello
```

Note

`srun` blocks your terminal until the command finishes — unlike `sbatch` which returns immediately. Use it for quick tests only, not long computations.

7.6 Checking job efficiency with seff and sacct

```
$ seff 30508122
Job ID: 30508122
State: COMPLETED (exit code 0)
Job Wall-clock time: 00:00:02
Memory Utilized: 19.67 MB
Memory Efficiency: 0.96% of 2.00 GB
```

This job used only 19.67 MB of the 2 GB requested. For the next run, request `--mem=100M` instead.

```
$ sacct              # show all recent jobs
$ sacct -j 30508122  # details for one specific job
```

Tuning workflow

1. First run: request generously (e.g. `--mem=8G`, `--time=02:00:00`)
2. Run `seff JOBID` after it finishes
3. Set future requests to actual usage + 20–30% buffer

8 Loading Software with Modules

Goal of this Section

Load the correct version of Python (and other software) on the cluster before running jobs.

Intuition

Thousands of researchers use the cluster with different software needs. If everyone's software was loaded at once, there would be version conflicts. The **module system** lets each user load exactly the software they need, at the version they need.
Think of it like a library: the books (software) are all on the shelves, but you have to check them out (`module load`) before you can use them.

Software is not loaded by default

When you log in, almost no software is pre-loaded. Even Python may not be the version you expect. Always explicitly load what you need — especially inside job scripts.

8.1 Exploring available software

```
$ module avail              # list all available software
$ module avail python       # search for python specifically
$ module spider python      # find all versions with details
$ module list               # show what is currently loaded
```

8.2 Loading and unloading

```
$ module load python/3.11.5    # load a specific version
$ python --version             # confirm
Python 3.11.5

$ module unload python         # unload
$ module purge                 # unload all non-core modules
$ which python                 # confirm which python is active
```

Always use a specific version

```
module load python/3.11.5   # specific - reproducible
module load python           # default - may change over time
```

If the cluster is updated and the default Python version changes, scripts using just `module load python` may break silently. Pinning to a version keeps your results reproducible.

Load modules inside your job script

What you load on the login node is **not** available on compute nodes. Every job script must load its own modules:

```
#!/bin/bash
#SBATCH ...
module load python/3.11.5    # REQUIRED in every job script
python my_script.py
```

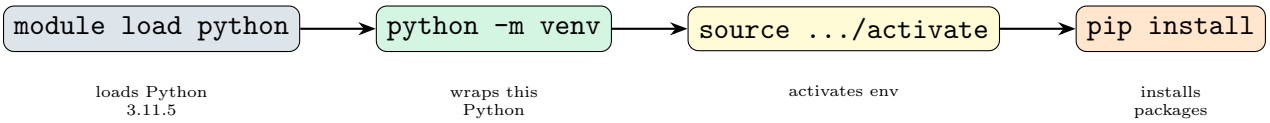
9 Running Python on the Cluster

Goal of this Section

Set up a Python virtual environment, install packages, write a script, and run it on a compute node.

Intuition

A **virtual environment** is an isolated Python installation for your project. It lets you install packages (like numpy or PyTorch) without affecting other users or other projects. Think of it as a dedicated toolbox for one project.
On the cluster, `venv` is the preferred tool (not conda). Unlike conda — which downloads a fresh Python — `venv` *wraps* the Python that is currently loaded. This is why you must `module load python` *before* creating the environment.



9.1 Step 1 — Create the environment

Try it yourself

```
$ module load python/3.11.5
$ python -m venv /scratch/yourUsername/my_env
```

Where to put your environment

```
# Good - permanent, backed up
$ python -m venv
   /home/yourUsername/projects/def-yourPI/envs/my_env

# OK for testing only - purged after 60 days!
$ python -m venv /scratch/yourUsername/my_env
```

9.2 Step 2 — Activate and install packages

Try it yourself

```
$ source /scratch/yourUsername/my_env/bin/activate
(my_env) $ pip install numpy
Successfully installed numpy-2.4.2+computecanada

(my_env) $ python -c "import numpy; print(numpy.__version__)"
2.4.2

(my_env) $ deactivate
```

When the environment is active, your prompt shows `(my_env)` at the start. After `deactivate`, packages are no longer available — confirmed by:

```
$ python -c "import numpy"
ModuleNotFoundError: No module named 'numpy'   # expected!
```

DRAC's wheel cache

Common packages like numpy, scipy, and PyTorch are pre-built for the cluster in a local cache. Running `pip install numpy` installs instantly from this cache rather than

downloading from the internet.

9.3 Step 3 — Write a Python script

Try it yourself

Make sure you are in your working directory first, then create the script:

```
$ cd /scratch/yourUsername/hpc-tutorial
$ nano test_script.py
```

```
import numpy as np

print("Python is working on the cluster!")
arr = np.array([1, 2, 3, 4, 5])
print(f"Array: {arr}")
print(f"Mean: {np.mean(arr)}")
print(f"numpy version: {np.__version__}")
```

9.4 Step 4 — Write the job script

Save the following as `python-job.sh` (use `nano python-job.sh`):

```
#!/bin/bash
#SBATCH -J python_test
#SBATCH --time=00:05:00
#SBATCH --mem=1G
#SBATCH --ntasks=1
#SBATCH --account=def-kanj241_cpu

module load python/3.11.5
source /scratch/yourUsername/my_env/bin/activate
python /scratch/yourUsername/hpc-tutorial/test_script.py
echo "Job done!"
```

Never use ~ in SLURM scripts

SLURM does not expand `~`. Always use full absolute paths confirmed with `pwd`. The `source` command for your virtual environment must also use the full path.
Because `pwd` inside your working directory shows `/home/yourUsername/scratch/hpc-tutorial`, you might be tempted to use that path in a job script — but the real path (and the one to use) is

`/scratch/yourUsername/hpc-tutorial`. See the symlink note in "Before You Begin".

9.5 Step 5 — Submit and check output

Try it yourself

```
$ sbatch python-job.sh
Submitted batch job 30508122

$ squeue -u yourUsername
JOBID      NAME          ST  TIME_LEFT
30508122   python_test   PD  5:00

$ cat slurm-30508122.out
Python is working on the cluster!
Array: [1 2 3 4 5]
Mean: 3.0
numpy version: 2.4.2
Job done!
```

10 Transferring Files with scp

Goal of this Section

Upload files from your Windows laptop to the cluster and download results back to your laptop.

Intuition

`scp` (secure copy) works exactly like copying a file on your laptop — except one end of the copy is on a remote machine. The syntax is always: `scp source destination`, where the remote side uses the format `username@hostname:path`.

What does `scp` stand for?

Secure **C**opy — uses the same encrypted SSH connection as your terminal session.

Note

Run `scp` from your **local PowerShell or Command Prompt window**, not from the SSH session. `scp` requires Duo authentication just like SSH.

10.1 Uploading to the cluster

```
# Upload a single file
scp "C:\Users\yourWindowsUsername\script.py"
   yourUsername@fir.alliancecan.ca:/scratch/yourUsername/

# Upload a whole folder
scp -r "C:\Users\yourWindowsUsername\my_folder"
   yourUsername@fir.alliancecan.ca:/scratch/yourUsername/
```

Why not `~/scratch` in scp?

On your local machine, `~` expands to your *local* home directory, not the cluster's. Always use the full remote path (`/scratch/yourUsername/`) in `scp` commands to avoid uploading to the wrong place.

10.2 Downloading from the cluster

```
# Download a single file
scp
   yourUsername@fir.alliancecan.ca:/scratch/yourUsername/results.txt
   "C:\Users\yourWindowsUsername\"

# Download a whole folder
scp -r
   yourUsername@fir.alliancecan.ca:/scratch/yourUsername/results/
   "C:\Users\yourWindowsUsername\"
```

10.3 Windows line endings — dos2unix

Files created on Windows have a hidden `\r` character at the end of every line. On Linux this shows up as `^M`:

Try it yourself

```
$ cat -A example-job.sh
#!/bin/bash^M$
#SBATCH -J my_test_job^M$
```

```
#SBATCH --time=00:05:00^M$      <- ^M means Windows line
   ending

$ dos2unix example-job.sh
dos2unix: converting file example-job.sh to Unix format...

$ cat -A example-job.sh
#!/bin/bash$
#SBATCH -J my_test_job$
#SBATCH --time=00:05:00$        <- clean!
```

⚠ Windows line endings break scripts

For plain text files, `^M` is harmless. But for Python scripts and SLURM job scripts, it causes errors like `/bin/bash^M: bad interpreter`. Always run `dos2unix` on any script you transfer from Windows. Alternatively, write your scripts in VS Code (which defaults to Unix line endings) or PyCharm (set `File` → `File Properties` → `Line Separators` → `LF`).

10.4 Large transfers — bundle first with tar

Transferring thousands of small files is slow because each file requires a separate network handshake. Bundle them into a single compressed file first.

On Windows (PowerShell): You must `cd` into the folder *containing* the folder you want to bundle — not into the folder itself — before running `tar`:

```
# Navigate to the PARENT of the folder you want to bundle
cd C:\Users\yourWindowsUsername\Downloads

# Bundle the results folder into results.tar.gz
tar -czvf results.tar.gz results

# Then upload the single archive to the cluster
scp results.tar.gz
   yourUsername@fir.alliancecan.ca:/scratch/yourUsername/
```

⚠ You must be outside the folder you are bundling

If you are *inside* `results\` when you run `tar`, it will fail with "Couldn't visit directory". Always `cd` to the parent folder first.

On the cluster (to bundle job outputs before downloading):

```
$ cd /scratch/yourUsername/hpc-tutorial
$ tar -czvf results.tar.gz results/
```

To unpack the archive after downloading:

```
# On Windows PowerShell
tar -xzvf results.tar.gz

# On the cluster
$ tar -xzvf results.tar.gz
```

For large or frequent transfers, use `rsync` instead of `scp`:

```
rsync -rvzP local/folder/
   yourUsername@fir.alliancecan.ca:/scratch/yourUsername/
```

Flags: `r` recursive, `v` verbose, `z` compress, `P` show progress and resume if interrupted.

11 Running GPU Jobs

Goal of this Section

Find what GPUs are available on Fir, request one correctly, and train a real deep learning model.

Intuition

GPU jobs require one piece of information you cannot guess: the exact **GRES name** of the GPU. Always look it up fresh with `sinfo` before writing a job script — GPU configurations on the cluster can change. The account names (`def-kanj241_cpu` and `def-kanj241_gpu`) are fixed for your lab and do not need to be looked up.

11.1 Step 1 — Find available GPUs

Try it yourself

List all GPU partitions and what they offer:

```
$ sinfo -o "%P %G" | grep gpu
gpubase_interac  gpu:nvidia_h100_80gb_hbm3_3g.40gb:4,...
gpubase_bynode_b1 gpu:h100:4(S:0)
gpubase_bygpu_b1 gpu:h100:4(S:2-3)
gpubase_bygpu_b1 gpu:nvidia_h100_80gb_hbm3_3g.40gb:4,...
```

```
... (many more partitions)
```

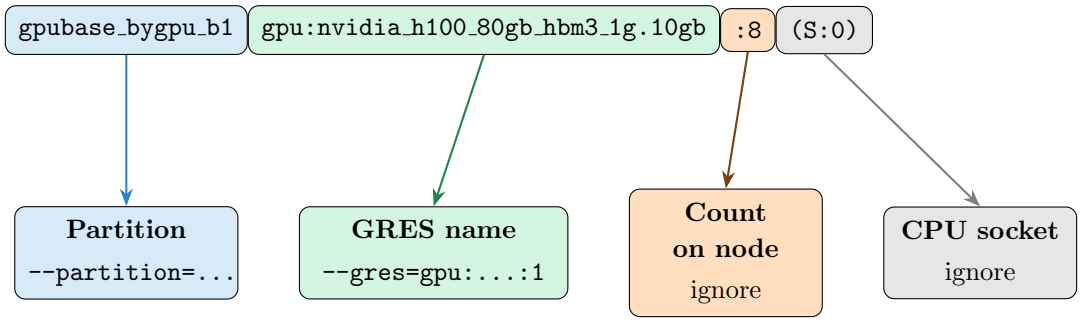
To narrow results to a specific partition, add a second `grep`:

```
$ sinfo -o "%P %G" | grep gpu | grep gpubase_bygpu_b1
gpubase_bygpu_b1 gpu:h100:4(S:2-3)
gpubase_bygpu_b1 gpu:nvidia_h100_80gb_hbm3_3g.40gb:4(S:2-3),...
gpubase_bygpu_b1 gpu:nvidia_h100_80gb_hbm3_3g.40gb:4(S:0),...
```

Why not use the fancier pipeline?

An earlier version of this command used `tr` and `sed` to split each line into one GPU per line. The problem is that `sinfo` only prints the partition name on the *first* GPU in each group — the rest get a blank left side. After splitting, those blank lines no longer match a `grep` filter, so GPU types get silently dropped. The simpler two-`grep` approach above keeps every line intact and filters correctly.

How to read each line:



The number after the GRES name (e.g. `:8` or `:4`) is how many of that GPU type exist on the node — that is the cluster's inventory, not something you set. In your `#SBATCH` line you replace it with how many *you* want, which is almost always `:1`:

```
# sinfo shows:   gpu:nvidia_h100_80gb_hbm3_1g.10gb:8   <- 8
   available on node
# You request:   --gres=gpu:nvidia_h100_80gb_hbm3_1g.10gb:1  <-
   you want 1
```

The `(S:i-j)` suffix refers to which CPU *socket(s)* the GPU is physically connected to on the motherboard. `S:0` means socket 0, `S:2-3` means sockets 2 and 3. This affects memory bandwidth between the CPU and GPU for highly optimised code, but for most jobs you can ignore it entirely — SLURM handles the assignment automatically.

GPU configurations can change

The partitions and GPU types shown by `sinfo` reflect the cluster's current hardware and configuration. DRAC admins can add, remove, or reconfigure them. Always run `sinfo` fresh — never hardcode values from a previous session without verifying.

Fir has NVIDIA H100 80GB GPUs available in four configurations:

GRES name	Memory	Use for
`nvidia_h100_80gb_hbm3_1g.10gb`	10 GB	small models, testing
`nvidia_h100_80gb_hbm3_2g.20gb`	20 GB	medium models
`nvidia_h100_80gb_hbm3_3g.40gb`	40 GB	large models
`h100`	80 GB	very large models

Tip

Always start with the smallest slice that fits your model. Smaller requests are scheduled faster and cost less allocation. A 10 GB slice is more than enough for most image classification tasks.

11.2 Step 2 — CPU vs GPU accounts

On DRAC, CPU and GPU resources use **separate accounts**. You do not need to look these up — they are fixed for your lab:

Account	Use for
`def-kanj241_cpu`	CPU jobs (`sbatch` without `--gres=gpu`)
`def-kanj241_gpu`	GPU jobs (`sbatch` with `--gres=gpu:...`)

Always use the _cpu or _gpu suffix

Using `def-kanj241` (without _cpu or _gpu) gives: `Invalid account or account/partition combination`. Always use the specific sub-account.

11.3 Step 3 — Install PyTorch and write the GPU job script

Before writing the job script, install PyTorch into your virtual environment on the login node:

Try it yourself

```
$ module load python/3.11.5
$ source /scratch/yourUsername/my_env/bin/activate
(my_env) $ pip install torch torchvision
```

```
Successfully installed torch-2.11.0+computecanada
   torchvision-0.26.0+computecanada
(my_env) $ deactivate
```

Packages install instantly from DRAC's local wheel cache — no internet download needed.

Which CUDA version to load?

Run `module avail cuda` to see what is available on Fir. At the time of writing, `cuda/12.6` is the default. Use `module load cuda` without a version to get the default, or pin to a specific version like `module load cuda/12.6`.

The training script — cifar10_train.py

Make sure you are in your working directory, then create the file:

```
$ cd /scratch/yourUsername/hpc-tutorial
$ nano cifar10_train.py
```

Try it yourself

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms

# Check GPU availability --- always print this first!
device = torch.device("cuda" if torch.cuda.is_available()
   else "cpu")
print(f"Using device: {device}")

# Download and load CIFAR-10 (downloads automatically on
   first run)
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

trainset = torchvision.datasets.CIFAR10(
    root='./data', train=True, download=True,
       transform=transform)
```

```
trainloader = torch.utils.data.DataLoader(
    trainset, batch_size=64, shuffle=True, num_workers=2)

testset = torchvision.datasets.CIFAR10(
    root='./data', train=False, download=True,
       transform=transform)
testloader = torch.utils.data.DataLoader(
    testset, batch_size=64, shuffle=False, num_workers=2)

# Define a simple CNN
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
        self.pool  = nn.MaxPool2d(2, 2)
        self.fc1   = nn.Linear(64 * 8 * 8, 512)
        self.fc2   = nn.Linear(512, 10)
        self.relu  = nn.ReLU()

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        x = x.view(-1, 64 * 8 * 8)
        x = self.relu(self.fc1(x))
        return self.fc2(x)

model     = SimpleCNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Training loop
epochs = 5
for epoch in range(epochs):
    running_loss = 0.0
    for inputs, labels in trainloader:
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
```

```
    # Evaluate on test set
    correct, total = 0, 0
    with torch.no_grad():
        for inputs, labels in testloader:
            inputs, labels = inputs.to(device),
               labels.to(device)
            _, predicted = torch.max(model(inputs), 1)
            total   += labels.size(0)
            correct += (predicted == labels).sum().item()

    print(f"Epoch {epoch+1}/{epochs} | "
          f"Loss: {running_loss/len(trainloader):.3f} | "
          f"Accuracy: {100*correct/total:.1f}%")

print("Training complete!")
torch.save(model.state_dict(), "cifar10_model.pth")
print("Model saved to cifar10_model.pth")
```

Now write the job script (use `nano gpu-job.sh`):

```
#!/bin/bash
#SBATCH -J cifar10_training
#SBATCH --time=00:30:00
#SBATCH --mem=8G
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --gres=gpu:nvidia_h100_80gb_hbm3_1g.10gb:1
#SBATCH --partition=gpubase_bygpu_b1
#SBATCH --account=def-kanj241_gpu

module load python/3.11.5
module load cuda
source /scratch/yourUsername/my_env/bin/activate
python /scratch/yourUsername/hpc-tutorial/cifar10_train.py
echo "Job done!"
```

11.4 Step 4 — Confirm the GPU is being used

Always add this at the top of your Python script:

```
device = torch.device("cuda" if torch.cuda.is_available() else
   "cpu")
print(f"Using device: {device}")
```

If the output shows `Using device: cuda` — the GPU is active. If it shows `cpu` — the module was not loaded or the GRES was not granted.

11.5 Real result — CIFAR-10 on Fir

```
$ cat slurm-30511085.out
Using device: cuda
Epoch 1/5 | Loss: 1.301 | Accuracy: 64.2%
Epoch 2/5 | Loss: 0.910 | Accuracy: 68.8%
Epoch 3/5 | Loss: 0.720 | Accuracy: 71.5%
Epoch 4/5 | Loss: 0.559 | Accuracy: 71.8%
Epoch 5/5 | Loss: 0.408 | Accuracy: 73.0%
Training complete!
Model saved to cifar10_model.pth
Job done!
```

Loss dropped from 1.301 to 0.408 and accuracy reached 73% in 5 epochs — trained on a real H100 GPU on Fir.